

Names, Not Rules: Diagnosing Memorisation in Language Model Knowledge Graph Link Prediction

Lynda Lagh

August 19, 2026

DRAFT — NOT FOR CIRCULATION. Values marked * were produced before two corrections and must be re-measured: (i) `rank.py` did not apply the permutation, so every condition S ranking figure was computed on the unpermuted graph; (ii) type tags were induced from the graph under test and are now taken from YAGO’s own class assertions, so conditions C, D, E and G must be rebuilt. Values marked ?? have not been measured. Delete `\pvbanner` and the `\pv` colouring once neither remains.

Abstract

Instruction tuning is now the standard way to adapt large language models to knowledge graph completion, and reported accuracies are high enough that the task is often treated as solved. We ask what those accuracies measure. Taking an early and widely cited system in this line as an object of study rather than as a baseline, we add a control it never ran: every entity name in the graph is replaced by an opaque identifier, so that relational structure survives while surface form does not. Reporting accuracy on the original and anonymised test sets as a pair splits a single figure into a memorisation term and a residual knowledge term. Evaluated as link prediction, where the chance level is an order of magnitude below the classification floor, **94.0*** % of above-chance skill on YAGO3-10 is entity surface form. A permuted-name control localises this further: **71.5*** points of it come from the binding between a name and a graph node rather than from the readability of the text. A familiarity split places that binding in the pretrained backbone rather than in the fine-tuning set. Type text drawn from an independent source does not substitute for names, and supplying it alongside names costs **7.3*** accuracy points; neither does neighbourhood context, once the context block is prevented from restating the answer. Results are reported for a single 1.5-billion-parameter backbone at one seed. The diagnostic requires no change to the method under test, and the code is released.

Keywords— knowledge graph completion, link prediction, large language models, instruction tuning, memorisation, evaluation methodology

1 Introduction

Knowledge graphs are queried as if they were complete, and they are not. Over 70 % of person nodes in Freebase lack a birthplace [11], and the systems built to fill such gaps are increasingly language models adapted by instruction tuning. An early and widely cited system in this line reports 95.5 % triple classification accuracy on WN11 against a chance level of 50 % [1, 2], and subsequent work

has treated that figure as a baseline to exceed rather than as a quantity to interpret.

A high number on this benchmark admits two explanations. The model may have acquired a relational rule, so that it can judge unfamiliar facts, or it may have learned an association between entity names and labels, so that it can judge only familiar ones. Both produce the same accuracy. The distinction matters because the entities that motivate completion are new, thin and long-tailed: precisely those for which a memorised surface form is worthless.

The two explanations differ in one observable way. A relational rule survives the replacement of entity names by opaque identifiers; a name-to-label association does not. We therefore train the same pipeline twice, once on the original graph and once on an anonymised copy, and report accuracy on both test sets as a pair. The difference between them is the share of performance carried by surface form.

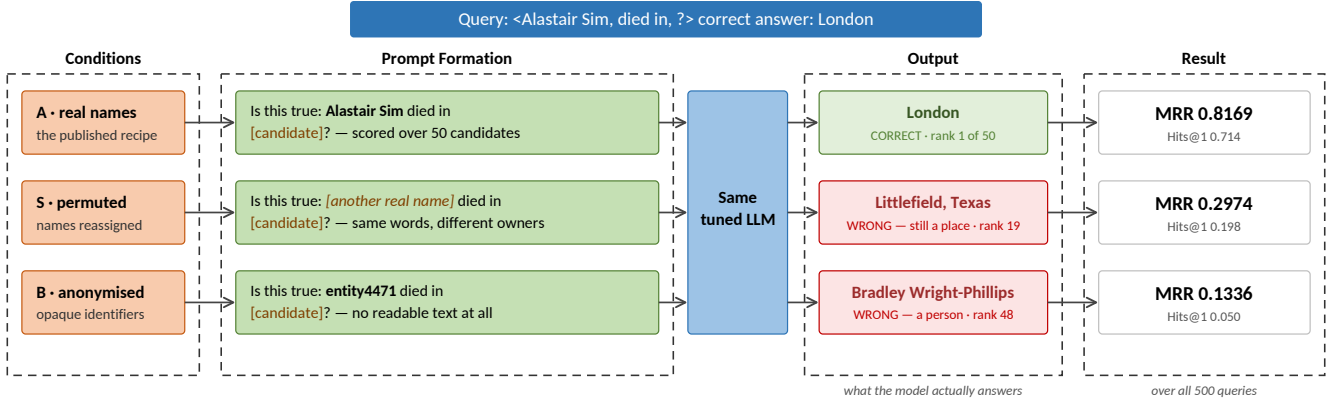
This paper makes four contributions.

A decomposition rather than a score. We split above-chance performance into format compliance, surface-form memorisation, and residual relational knowledge, and argue that the pair (accuracy, gap) should be the minimum unit of reporting for a completion system, in the way that a mean is not reported without a dispersion.

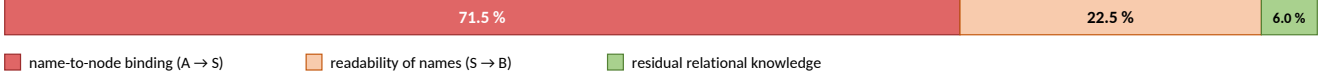
A seven-condition grid varying one factor at a time — names, type information, negative hardness, negative count — with the interpretation of every outcome recorded in the released code beside the grid itself.

Three instruments that fail differently. Permuted real names, a seen and unseen familiarity split that leaves names intact, and calibration, which reads the magnitude of the model’s score where ranking reads only its order. Agreement between instruments with no shared machinery is harder to dismiss than any one of them.

The diagnostic carried out on link prediction. Triple classification completes nothing, and its chance level of 0.5 lets a model that collapses onto one answer score exactly at chance while having learned nothing, a failure we observe in three conditions. Scoring candidates and sorting them moves chance to $\text{MRR} = 0.0900$ under the 50-way protocol. The principal result of this paper is therefore a link prediction result.



Decomposition of above-chance skill (50-way filtered, chance MRR = 0.0900)



The three arms differ only in entity surface form: relations, graph structure, adaptation recipe and token budget are identical. The **gap between arms** is the measurement, not any single accuracy. Under S the model still returns a **place** — the type survives, the identity does not; under B it returns a person, so readability went too.

Memorisation share = $(\text{MRR}_A - \text{MRR}_B) / (\text{MRR}_A - \text{chance}) = 94.0\%$ on YAGO3-10. [another real name] = a different entity's real name, under a fixed derangement.

Figure 1: The diagnostic. Three arms differ only in entity surface form; the relations, the graph structure, the adaptation recipe and the token budget are identical, and the same tuned model scores all three. The measurement is the *gap* between arms, not any single accuracy. Fifty-way filtered link prediction on YAGO3-10, chance MRR = 0.0900 by (1). **The permuted row is provisional; see the draft banner.**

2 Related Work

2.1 Completion, enrichment, and task variants

We use *enrichment* for any operation that adds information to a graph, and *completion* for the subclass that adds edges over a fixed vocabulary of entities and relations. Completion is thus one enrichment operator among several; the distinction is rarely drawn in this literature, and it matters here because our results bear on the enrichment cases rather than on the benchmark cases.

Within completion, four task variants recur: triple classification (decide whether a stated triple holds), entity or link prediction (produce the missing argument), relation prediction, and triple set prediction [20]. They are not interchangeable. Their chance levels differ by orders of magnitude, and only the last three produce a completion.

A completion experiment is *transductive* when every test entity appears in the training graph and *inductive* when test entities are new [7]. The distinction governs how much memorisation is worth: in the transductive setting a memorised name-to-label association is directly useful, which is why a benchmark built that way cannot reveal its own dependence on one. Under the closed-world assumption every triple absent from the graph is treated as false, so generated negatives may be unrecorded true facts [6]; we return to this in Section 5.3.

2.2 Choice of protocol

The decomposition we report requires a stable, non-zero chance level, since performance above chance is its denominator. That requirement dictates the protocol.

Triple classification satisfies it trivially, with chance ex-

actly 0.5, but it completes nothing. Full-ranking link prediction is the task, but with $|\mathcal{E}|$ in the hundreds of thousands its chance level is approximately zero, leaving the denominator undefined, and it costs $|\mathcal{E}|$ forward passes per query. The N -way protocol resolves both. Sampling $N = 50$ candidates, as [8] and [9] do, yields link prediction while fixing chance at

$$\text{MRR}_{\text{chance}} = \frac{1}{N} \sum_{k=1}^N \frac{1}{k} = 0.0900. \quad (1)$$

We report both protocols on the same checkpoints and treat link prediction as primary, retaining classification because it is what the literature under study uses. Fifty-way Hits@K is not comparable to full-ranking Hits@K, and every caption states the protocol.

2.3 Instruction-tuned completion

KG-LLM [1, 2] fine-tunes LLaMA and ChatGLM with LoRA [5] on prompts derived from KG-BERT [3], and is the system we reconstruct. Knit [10] is its closest descendant, projecting KGE vectors into the token embedding space on the same four benchmarks. EGIT [11] adds embedding-guided hard negatives, and supplies the definition of hallucination in completion that our results quantify: predictions that are structurally implausible yet semantically plausible, arising from “over-reliance on the language model’s parametric knowledge rather than graph evidence.” Neither measures parametric reliance.

2.4 Context enrichment and prompting

CATS [8] supplies reasoning paths and type constraints, RealKGC [9] relation-constrained structure blocks, GS-

Table 1: Positioning. *Adapt* = modifies model weights; *Context* = adds text to the prompt; *Name ctrl* = reports a condition in which entity surface forms are removed or perturbed; *Cost* = reports token or compute cost alongside accuracy.

System	Task	Adapt	Context	Name ctrl.	Costs
KG-LLM [1]	TC, LP	✓	✓		
Knit [10]	TC, LP	✓	✓		
EGIT [11]	LP	✓	✓		
FLAME [19]	LP	✓			
struOKGC [13]	LP	✓	✓		
APE [16]	TC, RP		✓		
CATS [8]	LP		✓		
RealKGC [9]	LP		✓		
GS-KGC [15]	LP		✓		
KICGPTv2 [12]	LP		✓		
MKGL [18]	LP	✓			
KG-CF [17]	LP		✓	partial	
This work	TC, LP	✓	✓	✓	

TC = triple classification, LP = link prediction, RP = relation prediction. *Name ctrl.* counts a condition applied to the reported headline figure; KG-CF applies one to a single component.

KGC [15] subgraph neighbourhoods, KICGPTv2 [12] in-context demonstrations, struOKGC [13] relation-specific templates, APE [16] auto-generated prompts, and ToC [14] LLM-guided context for inductive relation prediction. All add context; none removes the entity name, so none can separate a context effect from a familiarity effect. KG-LLM itself belongs in this list: its largest single reported gain on YAGO3-10 comes from sampling $K = 5$ neighbouring entities into the prompt. We reproduce that mechanism in Section 3.5 rather than assume it.

2.5 Contamination, memorisation, and shortcut learning

KG-CF [17] is the closest prior use of the instrument. One ablation replaces the entity names inside its path trajectories with opaque labels, keeps the relations, and reports a drop that the authors read as evidence of leakage. It differs from ours in three ways. The query head and tail keep their names, so name-to-node binding is never removed. There is no permuted-name arm, so the drop confounds the loss of binding with the loss of readable text. And the result justifies a design decision rather than decomposing a headline figure against a chance floor. FLAME [19] asks a related question of *frozen* models, quantifying task-relevant information in intermediate representations with sliced mutual information; that instrument locates *where* label information sits, not *which* input property supplies it.

A model prefers a feature that is available, sufficient and cheap over one that is none of these [21]. Entity names on a transductive benchmark are all three.

Table 1 positions this work. The pattern is consistent: methods either adapt the model or enrich its context, both report accuracy, and neither controls for surface form. KG-CF is the single exception, and it applies the control to one component rather than to a headline figure.

3 Method

3.1 Decomposing performance above chance

We split performance above chance into three measured components:

$$\text{acc} - c = \underbrace{\phi}_{\text{format}} + \underbrace{\mu}_{\text{memorisation}} + \underbrace{\kappa}_{\text{residual}} \quad (2)$$

where c is the protocol’s chance level: 0.5 for classification, 0.0900 for 50-way link prediction by (1). Only c changes between protocols, which is what allows the two readings to be compared.

Format compliance ϕ is the accuracy recovered by reading one set of generations under a more permissive decision rule, that is, the share of apparent error that is an output-convention failure rather than a wrong answer. We read each set of generations under four rules: the strict and lenient substring rules of [1], a constrained rule, and a generation-free comparison of the probability assigned to the affirmative and negative tokens.

Memorisation μ is the gap

$$\mu = \text{acc}(\mathcal{T}) - \text{acc}(\mathcal{T}_{\text{anon}}), \quad (3)$$

where $\mathcal{T}_{\text{anon}}$ replaces every entity surface form with an opaque identifier while leaving relations and graph structure untouched.

Residual knowledge κ is what remains: $\text{acc}(\mathcal{T}_{\text{anon}}) - c$. It is the quantity a completion system needs in order to be useful on entities it has not seen.

3.2 The pipeline under study

We reconstruct the target system as eight stages, each of which is a decision an author must make and most of which are reported only in passing: (1) task variant; (2) negative construction; (3) enrichment of the prompt with entity and relation text; (4) prompt template; (5) adaptation method and its hyperparameters; (6) decoding; (7) the rule that converts a generation into a label; (8) the metric and its chance level. Our conditions perturb stages 2, 3 and 5, and our instruments perturb stages 7 and 8, which is where we find that much of the reported difference between systems lives.

Two stages deserve emphasis because they are rarely varied. Stage 7 is the decision rule; [1] uses substring matching, which fires on *know* and *cannot* and scores a refusal as a negative. Stage 8 is the chance level, which the N -way protocol changes by an order of magnitude (Section 2.2). Neither is a modelling choice, and both move the reported number.

3.3 Where the type information comes from

Half the conditions add a type tag to each entity, and the source of that tag decides what they can establish. We distinguish two, and use only the second.

An *endogenous* type is computed from the graph under test. The natural construction — and the one we used first — labels each entity by the (relation, argument-position) pair it occupies most often, giving tags such as

[wasBornIn : tail]. It is cheap, it needs no external resource, and it is circular: the tag is a summary of the very edges the model is asked to predict. The circularity is measurable. On our first YAGO3-10 negatives, the one-line rule *answer yes if the candidate tail’s tag names the query relation* scored 62.4% with no model at all (Section 4.2). A condition built on such tags cannot separate type reasoning from label leakage.

An *exogenous* type comes from a source that does not depend on the completion task. YAGO3-10 has one: YAGO [23] ships `rdf:type` assertions linking each entity to a leaf class of the WordNet hierarchy, so *Alastair.Sim* carries [actor] and *London* [city]. We take the `yagoSimpleTypes` theme of the YAGO 3 release, one class per entity, and treat an entity without an assertion as **OTHER** rather than guessing. Coverage on the 123,182 entities of YAGO3-10 is 99.9%, over 286 distinct classes, with 0.1% **OTHER**; the type file is written only if it covers 90% of the entity set, so a silent downgrade to induced tags cannot occur.

Two properties of that inventory bound what the typed conditions can show, and we report them rather than treat 99.9% coverage as sufficient. It is *concentrated*: the largest class alone covers 37.9% of entities and the top five 71.2%, and because YAGO’s WordNet and Wikipedia-category layers name the same notion differently (**people** and **person** are separate classes, alongside **artist**, **writer**, **scientist** and **officeholder**), roughly half of all entities carry some class meaning “a person”. And it is *mixed in provenance*: 57% of the labels come from the WordNet layer and 43% from a head-noun reduction of Wikipedia category names, which is exogenous but noisier. A null on condition C is therefore a statement about this inventory, not about semantic types in general.

Two properties are required of any tag we use. It must be *anonymisation invariant*: the tag is a class name, never an entity name, so `entity4471` [actor] discloses a category and not an identity, and conditions B and C differ in exactly one respect. And its residual leakage must be *measured, not assumed*: we regenerate the test negatives to be type-consistent — a corrupted tail is drawn uniformly from the entities observed as tails of the same relation — and re-run the one-line rule, admitting a condition only when that rule sits near its 0.5 floor and is not merely constant. The distinction matters: a rule that fires on every row of both classes also scores 0.5 on a balanced set, and reporting that as a cleared floor would be an artefact of the same kind the audit exists to catch.

WN11 has no such source. Its relations *are* its taxonomy (`_type_of`, `_member_holonym`), so any type derived for it is endogenous by construction; the audit in Section 4.2 confirms this. We therefore report no typed condition on WN11, and all type results in this paper are YAGO3-10 results.

3.4 Condition grid

Table 2 lists the seven conditions and Fig. 2 shows the prompts they produce. Each varies one factor from the baseline. The interpretation of each outcome is declared in the released code beside the grid that defines the conditions, so a result cannot be reassigned a meaning after it is seen; we make no claim to a registered protocol beyond

Table 2: Condition grid. Each row changes one factor from A. Types are exogenous throughout (Section 3.3).

	Names	Types	Isolates
A	real	no	baseline: the published recipe
B	anonymous	no	entity names
C	anonymous	yes	whether types can replace names
D	anonymous	yes	negative <i>hardness</i> (count fixed)
E	anonymous	yes	negative <i>count</i> (hardness fixed)
G	real	yes	do types help when names are present?
S	permuted	no	name-to-node <i>binding</i> , not readability

that.

One exception is material and we mark it rather than let the sentence above cover it. The declared readings for the permuted arm were $S \approx B$ (the binding carried the signal) and $S \approx A$ (names were never used). The outcome was neither: S fell strictly between them, and the three-way decomposition of Section 4.4 — binding, readability, residual — was written to accommodate it. That decomposition is therefore *exploratory*, and confirming it on a second graph is the first item of future work rather than a refinement.

3.5 Context variants

The conditions above change what the model is *trained* on. A second, orthogonal family changes what it is *asked*, and lets us reproduce the context mechanisms of Section 2 inside the same measurement frame. Beyond the plain prompt P0 we evaluate three: P5 prepends a one-sentence description of what the query relation means, hand-written and therefore exogenous; P6 prepends $K = 5$ facts about the head entity, which is KG-LLM’s own mechanism and the source of its largest reported YAGO3-10 gain; P7 prepends up to three relation chains linking the head to the candidate, in the manner of CATS and KG-CF.

A context block that mentions the answer measures nothing, and the obvious guard is insufficient. Excluding the target entity still admits the mirror fact: if the query is $(h, r, ?)$ with gold t , a neighbour edge (t, r^{-1}, h) states the answer outright in the opposite direction. Our guard therefore excludes four things from every block — the gold entity, every edge on the query relation, every edge on its inverse, and the query triple itself — and is validated by an assertion that no block over a held-out sample contains the gold string. The same assertion is run in the reverse direction to confirm the guard is not simply emptying the block. Inverse pairs are not assumed from the relation names: r' is treated as an inverse of r when at least half of the (h, r, t) edges have a matching (t, r', h) edge in the training graph. A guard that drops only r itself is correct on a graph that stores mirrors under one name and silently incomplete on one that does not, and which of those holds is a property of the release rather than of the method.

Two readings are available and they differ in strength. Evaluating a P0-trained checkpoint under P5–P7 is cheap but asymmetric: an increase is interpretable, a decrease may be distribution shift rather than inability, since the model was tuned on one format and shown another. We therefore also *train* a condition on P6, which removes the asymmetry at the cost of one further run.

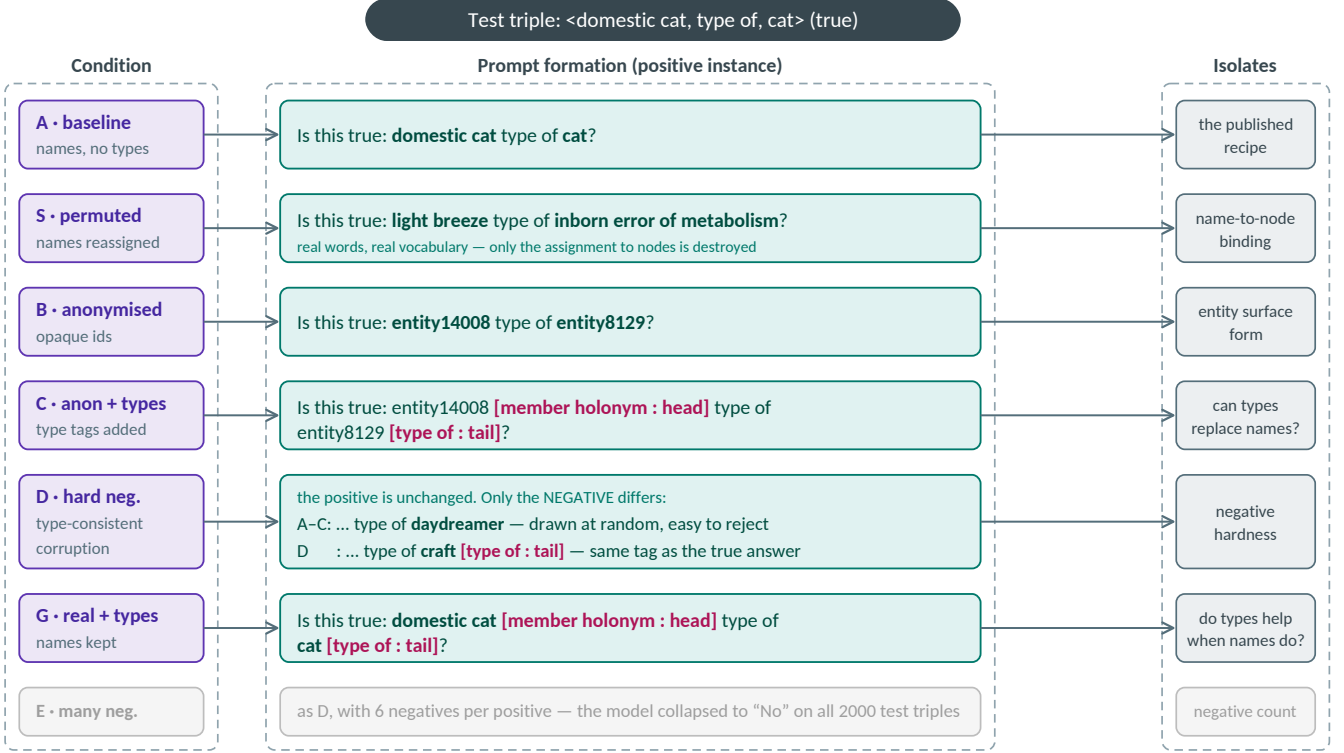


Figure 2: The condition grid instantiated on one test triple. Prompts are verbatim output of the released builder. Type tags are *exogenous*: they are YAGO’s own `rdf:type` assertions into the WordNet class hierarchy, not statistics of the edges under test (Section 3.3). Condition E is greyed because it produced a degenerate classifier. **Figure to be regenerated from the exogenous builder.**

3.6 Corroborating instruments

The principal objection to an anonymisation experiment is that replacing a name with `entity4471` removes readability as well as identity, so the collapse would be uninformative. Condition S answers this with a measurement. Every name in the graph is kept and only the assignment of names to entities is deranged, so vocabulary, token distribution, name lengths and readability are all preserved while the name-to-node binding is destroyed. If accuracy also collapses under S, the collapse cannot be attributed to unreadable identifiers.

The *familiarity split* partitions test triples by how many of their two entities were touched by the training sample. Names are intact in every bucket, so this instrument shares no mechanism with anonymisation. Because the buckets have different positive rates, we report balanced accuracy; the raw figure credits an always-affirmative predictor with a familiarity effect it does not have.

Calibration reads the magnitude of the model’s score, where ranking reads only its order, and is therefore invariant to different things.

3.7 From classifier to ranker

Scoring each candidate t for a query $(h, r, ?)$ by $P(\text{Yes} | h, r, t)$ and sorting yields an ordering, hence Hits@ K and MRR, without retraining. The principle is that of KG-BERT [3], which ranks entities by a classifier score. It also sidesteps a practical limitation of the generative formulation: decoding yields a set rather than an order, so

ranking metrics are not directly available from the generations themselves.

4 Experiments

4.1 Setup

Datasets. YAGO3-10 (123,182 entities, 37 relations) is primary and WN11 (38,588 entities, 11 relations) secondary. YAGO3-10’s relations are strongly typed — `wasBornIn` maps a person to a place, `playsFor` a person to a club — which is the structure the type conditions require, and it is the only one of the two with an exogenous type source (Section 3.3). Running both is a decomposition rather than a replication: memorising on WN11 is lexical, on YAGO3-10 world-factual.

Model and tuning. Qwen2.5-1.5B-Instruct [24] with LoRA [5], $r = 8$, $\alpha = 16$, on the query and value projections, two epochs over 20,000 instances, effective batch 32, learning rate 3×10^{-4} . The size is matched to the sub-7B ablations of [8] and [15]. Weights are loaded in fp32 with fp16 compute and SDPA attention; fp16 weights with eager attention produce non-finite logits on this backbone, which surfaces as a training loss of exactly zero and resembles a completed run.

Protocol. Test sets are balanced, so accuracy equals balanced accuracy and chance is 0.5. Ranking uses 500 queries, 50-way, filtered against train $\cup$ valid $\cup$ test, giving 25,000 scored candidates per condition. All figures use the generation-free decision rule, which is format-independent by construction. A preflight stage refuses to start a run

whose test labels are missing or unbalanced, whose type coverage is below threshold, whose measured tag-only floor is unrecorded, or whose permuted arm does not differ from its baseline; each of those checks corresponds to a failure that occurred silently during this work.

4.2 Two artefacts removed before use

Type-tag leakage. A type tag can answer the question on its own, and how much it does is a property of the tag source and the negatives together, so we measure it for each combination we use. With *induced* tags and randomly corrupted negatives, the one-line rule *if the candidate tail’s tag names the query relation, answer yes* scored 62.4% with no model at all: 12.4 points of free accuracy on a prompt of our own design. Regenerating the test negatives type-consistently — drawing each corrupted tail from the entities the same relation is observed to take — reduced it to 51.3%. Moving to the exogenous inventory reduces it again, to 50.7% with a firing separation of 0.014 between positives and negatives, and conditions C and G return the identical figure, which is the empirical form of the invariance the design requires. The residue is concentrated in a few small relations (**happenedIn** 59.5%, $n = 42$; **isCitizenOf** 56.7%, $n = 30$), so we report per-relation accuracy alongside the aggregate.

The exogenous rule is *tail class equals the relation’s most common tail class*, and choosing it is not arbitrary. The obvious alternative — tail class lies in the relation’s observed range — is *vacuous* on this test set: the negative sampler draws from that range, so the rule fires on 100% of both classes and scores exactly 0.5. A constant rule and a cleared floor produce the same number on a balanced set, and only the firing rates distinguish them. We therefore report the firing rates with every floor and treat a saturated rule as unmeasured rather than clean.

The same audit on WN11 returns 56.8% with a separation of 0.136, concentrated in the taxonomic relations (**_type_of** 72.5%, **_member_holonym** 71.4%), which are exactly those a type-augmentation method would claim to help, and is the empirical form of the argument in Section 3.3 that WN11 admits no exogenous type source. An unmeasured floor of this size is silently added to every typed result.

Mirror facts in context blocks. The neighbour and path blocks of Section 3.5 were first built with the conventional guard, which excludes the target entity. Auditing 300 blocks found the gold answer present in 24* of them, supplied by edges on the inverse of the query relation. With the four-part guard in place the same audit returns ?? of 300, while a complementary check confirms the blocks remain non-empty and reports the number of inverse pairs the detector found. A context result measured before this correction would have reported the leak as a gain.

4.3 Link prediction

All figures below are single runs of one backbone. Where a condition produced a degenerate classifier we say so and withhold it from interpretation.

Table 3 separates the conditions by an order of magnitude. The mean rank column states the collapse plainly: with real names the gold tail sits at rank 1.72 of 50; with

Table 3: Link prediction on YAGO3-10. 500 queries, 50-way filtered against a pool of 123,182 entities. Fifty-way Hits@ K is not comparable to full-ranking Hits@ K . The untuned row is the same backbone with no adapter and establishes what the pretrained model alone supplies; the P5–P7 rows re-score the condition A checkpoint under the context blocks of Section 3.5.

		MRR	H@1	H@3	H@10	MR
<i>tuned, plain prompt</i>						
real names	A	0.8169	0.714	0.898	0.994	1.72
permuted names	S	0.2974*	0.198*	0.306*	0.498*	15.65*
anonymised	B	0.1336	0.050	0.114	0.272	23.05
anon. + types	C	0.0945*	0.026*	0.068*	0.194*	26.71*
<i>untuned backbone, no adapter</i>						
real names	A	??	??	??	??	??
anonymised	B	??	??	??	??	??
<i>condition A re-scored under context</i>						
relation text	P5	??	??	??	??	??
$K=5$ neigh.	P6	??	??	??	??	??
paths	P7	??	??	??	??	??
<i>trained on P6</i>						
$K=5$ neigh.	A	??	??	??	??	??
chance		0.0900	0.020	0.060	0.200	25.50

Table 4: Decomposition of above-chance ranking skill on YAGO3-10. The three terms sum to 100% by construction; how the mass distributes across them does not. Every row depends on $m(S)$ and is provisional.

Component	ΔMRR	Share
total above chance, $m(A) - c$	0.7269	100.0%
names permuted, $A \rightarrow S$	0.5195*	71.5%*
names removed, $S \rightarrow B$	0.1638*	22.5%*
residual, $m(B) - c$	0.0436	6.0%
memorisation share	0.6833	94.0%

names permuted it falls to **15.65***, anonymised to 23.05, and with type tags added to the anonymised prompt to **26.71***, beyond the 25.5 that random ordering produces. Condition C fails to help; whether it actively harms is not resolved at this sample size (see below).

The untuned rows separate two things that a tuned-only table confounds. Any above-chance ranking they show is supplied by pretraining and by the 50-way candidate pool, not by our adaptation, and the tuned–untuned difference is the part attributable to instruction tuning at all.

The context rows test whether the mechanisms of Section 2 recover what anonymisation removes. Because a decrease on a P0-trained checkpoint is not interpretable (Section 3.5), only the P6 training row licenses a two-sided reading.

4.4 Decomposition

Because S preserves the vocabulary and destroys only the binding between a name and its entity, the conditions partition above-chance skill into three additive terms. Writing $m(\cdot)$ for MRR and c for chance,

$$m(A) - c = \underbrace{[m(A) - m(S)]}_{\text{binding}} + \underbrace{[m(S) - m(B)]}_{\text{readability}} + \underbrace{[m(B) - c]}_{\text{residual}}. \quad (4)$$

Table 5: Triple classification, 2000 balanced test triples, chance 0.5. TPR and TNR are recovered from accuracy and positive rate; AUC is threshold-free and therefore survives the collapse that destroys accuracy. Rows marked † are degenerate and are excluded from interpretation. p is a McNemar exact test against condition A on the same triples.

		Acc.	Pos.	TPR	TNR	F1	AUC
<i>YAGO3-10</i>							
baseline	A	0.7465	0.658	0.904	0.589	??	??
trained anon.	B	0.6655	0.310	0.475	0.856	??	??
anon. + types	C	0.5985*	0.760*	0.858*	0.339*	??	??
hard negatives	D	0.6410*	0.634*	0.775*	0.507*	??	??
many neg.†	E	0.5010*	0.001*	0.002*	1.000*	??	??
real + types	G	0.6735*	0.779*	0.952*	0.395*	??	??
permuted names	S	0.6030*	0.346*	0.449*	0.757*	??	??
<i>WN11</i>							
baseline	A	0.9265	0.488	0.914	0.939	??	??
trained anon.	B	0.7255	0.590	0.815	0.636	??	??

Three readings follow. First, the permuted-name control carries the argument: every name remains real and readable, only the assignment is deranged, and performance still falls by **71.5*** % of all above-chance skill. What the model uses is therefore predominantly which string sits on which node, not the readability of the strings, though readability accounts for a further **22.5*** % and is not negligible. Second, the residual is small in absolute terms. After both controls, $MRR = 0.1336$ against a floor of 0.0900. Note that the memorisation share, $[m(A) - m(B)]/[m(A) - c] = 94.0\%$, does not depend on S; only its division into binding and readability does. Third, type text does not substitute for names: C exceeds chance by **0.0045*** and ranks below B, on a test set whose tag-only floor was measured at 0.507 with a firing separation of 0.014, so the leak audited in Section 4.2 does not explain the result.

4.5 Triple classification

Table 5 shows why we do not lead with this protocol. Condition E, which raises the number of negatives per positive, produced a model whose $P(\text{Yes})$ never exceeds 0.5 on any of the 2000 test triples: it answers negatively throughout and scores **0.5010***, the negative base rate. Its nominal memorisation share is -19.0 , an artefact of dividing by a gap that does not exist. We report E as a failure of the recipe at that negative ratio and draw no conclusion from it about negative count. The AUC column is included for exactly this case: a model whose scores order the triples correctly but whose threshold has collapsed is distinguishable from one that has learned nothing, and accuracy alone cannot tell them apart.

The same pathology appears in the anonymised arms. On WN11, condition A evaluated on anonymised data answers affirmatively to 94.5 % of triples (TPR 0.946, TNR 0.056); on YAGO3-10, condition C’s anonymised arm answers affirmatively to **92.2*** %. In both cases the correct statement is that the model collapses onto a constant when names are removed, not that its accuracy fell to chance.

Two rows survive intact. Condition D shows that replacing random corruptions with type-consistent ones costs the baseline **10.6*** accuracy points ($0.7465 \rightarrow$

Table 6: Triple classification accuracy (%) on WN11. Selected rows of KG-LLM’s Table II [1], reproduced with their citations, plus ours. FB13 is omitted because we did not run it.

Method	WN11
TransE [4]	75.9
DistMult [1]	87.1
KG-BERT(a) [3]	93.5
KG-T5 [1]	72.8
LLaMA-7B (untuned) [1]	21.1
—LLaMA-13B (untuned) [1]	28.1
??KG-LLaMA-7B [1]	95.5
??KG-LLaMA-13B [1]	95.6
??KG-LLaMA2-13B [1]	96.6
ours, condition A (Qwen2.5-1.5B)	92.65
ours, condition B (anonymised)	72.55

Our backbone is 1.5B against their 7B and 13B, and our WN11 run had not converged when training stopped, so 92.65 is a floor. Three cells of this table must not be filled from our results: FB13, which we never ran; YAGO3-10 classification, where our negatives are generated and theirs ship with the benchmark; and link prediction, where they rank against all 123,182 entities and we against 50.

0.6410*): a substantial part of the headline figure was the ease of the negatives. Condition G shows that adding type text when names are already present costs **7.3*** points and drives the classifier toward the affirmative class (TPR **0.952***, TNR **0.395***).

The two protocols agree on every sign. Taking A against B evaluated on anonymised data (0.7465 vs 0.5080 , chance 0.5) gives an above-chance signal of 0.2465, of which 0.2385 is removed by anonymisation: a memorisation share of 96.8 %, against 94.0 % from ranking. Condition C agrees more sharply still: its anonymised accuracy is **0.5160*** against the measured tag-only floor of 0.507, a margin of **+0.009***, and in ranking it exceeds chance by **+0.0045***. Two protocols with different floors both place C at its floor.

4.6 Comparison with the published pipeline

Table 6 places our reconstruction inside KG-LLM’s own comparison table. The purpose is to establish that the pipeline we anonymise is the pipeline they describe, not a weakened stand-in: a diagnostic run on a broken reproduction would measure the breakage.

Two comparisons in that table are worth stating explicitly. The untuned LLaMA rows, at 21.1 and 28.1, are far below chance, which is a format failure rather than a knowledge failure and is the phenomenon our decision-rule sweep quantifies in Section 4.8. And the distance from KG-BERT’s 93.5 to KG-LLaMA2’s 96.6 is 3.1 points, while anonymisation removes 20.1 points from our own reconstruction: the effect this paper measures is larger than the margin the literature competes over.

4.7 Familiarity split

The untuned arm is a control, not a measurement: that model never saw the training split, so its $+0.0182$ is what the bucket construction produces on its own. It establishes that the two extreme buckets are not intrinsically of different difficulty.

Table 7: Familiarity split on WN11, balanced accuracy. The untuned arm is a control: it never saw the training set, so any gap it shows belongs to the buckets rather than to learning.

Bucket	n	Untuned	Tuned	Tuned anon.
both seen	438	0.6943	0.9255	0.5247
one seen	1063	0.6954	0.9275	0.5264
neither seen	499	0.6761	0.9219	0.5507
balanced gap		+0.0182	+0.0036	−0.0260
raw gap		+0.0964	+0.0031	−0.1760

Table 8: WN11 accuracy under four decision rules applied to identical generations. The spread is protocol, not capability.

Decision rule	Untuned	Tuned	Tuned anon.
strict	0.6775	0.9265	0.5325
lenient	0.6880	0.9265	0.5325
constrained	0.7475	0.9260	0.5255
generation-free	0.6920	0.9265	0.5325
unparseable	2.1 %	0.0 %	0.0 %
refusal scored negative	0.4 %	0.0 %	0.0 %

The tuned arm is the measurement, and it is +0.0036. Fine-tuning on a triple’s entities improves balanced accuracy on that triple by essentially nothing, while anonymisation removes 0.3940 from the same model on the same test set. The two differ by two orders of magnitude.

The conclusion is therefore sharper than “the model memorises”. What carries the accuracy is not recall of the fine-tuning set but an association between a surface form and a fact already present in the pretrained backbone. This bounds the most obvious alternative explanation — that the model is reciting its own fine-tuning data — at a balanced-accuracy effect of +0.0036 on this split. It is a null on one graph, not a proof, and a graph whose entities are absent from pretraining could behave differently.

The table also shows why balancing is not optional. Bucket positive rates are 0.607, 0.500 and 0.407, so a predictor that always answers affirmatively scores a +0.20 raw gap having learned nothing.

4.8 Format, calibration, and representations

On the untuned model the choice of decision rule moves accuracy by 0.0700 against an above-chance signal of 0.1920. More than a third of what a substring protocol reports as untuned capability is an artefact of how the output is read; 0.4 % of responses are refusals, which the lenient rule converts into negatives, penalising the epistemically correct behaviour. After tuning the spread collapses to 0.0005 and every response is parseable. Instruction tuning’s most reliable effect is on format, and a protocol that cannot separate format from knowledge will attribute the former to the latter. This is the mechanism behind the 21.1 % untuned LLaMA row of Table 6.

Calibration corroborates from a signal ranking does not use. Ranking uses the order of $P(\text{Yes})$ and is invariant to monotone rescaling; calibration uses its magnitude and is blind to order. When the binding is destroyed in condition S, mean confidence falls from 0.809 to **0.511***, near

indifference, while accuracy falls only to **0.603*** (expected calibration error [22] 0.062 for A, **0.092*** for S). The model is not merely less accurate without the binding; it registers that it is guessing.

Sliced mutual information between hidden states and labels, measured on the two YAGO3-10 arms, separates them only late: 0.00736 versus 0.00725 at layer 7, but 0.08239 versus 0.01683 at layer 21 and 0.07085 versus 0.01246 at layer 28. Removing entity names leaves early features untouched and removes roughly four-fifths of the label information present where a task decision would form.

5 Discussion and Conclusion

5.1 What the gap means

The ranking view makes the answer unambiguous: 94.0 % of above-chance skill disappears when surface forms are replaced. The permuted-name control determines how this must be worded. Because S retains every real name and destroys only the assignment, and because it alone accounts for **71.5*** % of the loss, the operative variable is the binding between a surface form and a specific node, not the presence of natural-language text. This is the distinction an anonymisation-only experiment cannot draw. The familiarity split then locates where the binding was acquired: not during fine-tuning, since withholding an entity moves balanced accuracy by +0.0036. In the vocabulary of [21], the name is available, sufficient and cheaper than the relational computation, so it is the feature that gets used.

5.2 Types, negatives, and context

Condition C sits at chance, at **0.0945*** against a floor of 0.0900. Its point estimate is below B, but a paired bootstrap over per-query reciprocal ranks does not separate the two ($C - B = -0.027*$, 95 % CI $[-0.063, +0.009]*$), so we claim only that types fail to substitute for names, not that they are actively harmful in the ranking view. The remaining explanation is capacity: LoRA at $r = 8$ may lack the room to install a type rule. It is no longer that the types are too weak to support one — they are YAGO’s own class assertions, external to the edges under test (Section 3.3), which is the strongest type signal this benchmark admits. A result obtained with induced tags would leave both explanations open; this one does not.

Condition G is worse than A by **7.3*** points, with the classifier pushed toward the affirmative class. Names are present in every condition that [8], [10] and [9] compare, so their module-level ablations cannot separate a type-reasoning effect from a token-quantity effect. Our G isolates that comparison and finds the addition harmful at this scale.

Negatives. Replacing random corruptions with type-consistent ones costs **10.6*** points, so a substantial fraction of the headline number is attributable to the ease of the negatives. This supports the reading that gains reported by [9] and [8] may owe more to their negative sampling than to their constraint modules, though we have not tested their systems. The count axis failed: raising

the number of negatives collapsed the model onto a constant answer, which bounds the recipe rather than the idea.

Context. The neighbour and path blocks are the mechanism by which the context-enrichment literature would answer this paper, and reproducing KG-LLM’s own $K = 5$ neighbours inside our frame tests it directly. Two properties of that test are worth separating from its outcome. It is only meaningful with the four-part guard of Section 3.5: 24* of 300 blocks contained the answer before it, and the leak would have appeared as a gain. And a gain under context is not by itself an answer to the diagnostic, since a block that names the head by its real name reintroduces exactly the binding condition B removes; the informative comparison is context added to the *anonymised* arm.

5.3 Threats to validity

Scale. A 1.5-billion-parameter model may memorise where a larger one generalises. Our conclusions are stated for this scale, matched to the sub-7B ablations of [8] and [15]; a scale sweep is future work.

Anonymisation could destroy more than names. This is the principal threat and the reason S exists. The outcome is intermediate, $m(S) = 0.2974^*$ against $m(A) = 0.8169$ and $m(B) = 0.1336$, which is the informative case: it separates readability (22.5* %) from binding (71.5* %).

Single run. Every figure is one training run at one seed, and ranking uses 500 queries, so no contrast here has a variance estimate and small ones should be read as provisional. Two independent runs of the WN11 conditions differed in final training loss by 0.0012 and 0.0005, which indicates a stable optimisation but does not bound the variance of MRR; only repeated seeds would. We rest the argument on the size of the principal effects — 0.8169 against 0.1336 — rather than on that stability.

One graph carries the headline. The decomposition rests on YAGO3-10 alone; WN11 contributes classification and familiarity results but has no permuted-name arm, and has no exogenous type source, so it contributes no typed condition either.

The type inventory bounds the typed conditions. Coverage is not the limitation — 99.9 % of entities carry a class — but concentration and provenance are. Half the entity set resolves to some notion of “person”, split across six near-synonymous classes by YAGO’s two labelling layers, and 43 % of the labels come from a head-noun reduction of Wikipedia categories rather than from WordNet. A null on C therefore bounds what *this* inventory can do; a curated, non-redundant type system could behave differently.

Candidate pools depend on the query budget. The 50-way pools are drawn from a single shared random stream, so they are a function of the query order and of the number of queries requested. Arms are comparable to each other only when ranked at the same budget, which every run here was; pools are not reproducible across a change of budget.

Two anonymised WN11 test sets exist and this paper quotes both. The decision-rule and familiarity tables report the tuned model at 0.5325 on anonymised WN11; the gap arithmetic elsewhere follows a later evaluation of the same checkpoint that returns 0.5010. The two anonymised

sets were built separately and carry different generated negatives, which is a 3.15-point difference in the arm that sits closest to the floor. The final version will report one of them; until then the WN11 residual should be read as 0.03*–0.00* rather than as a point estimate. The YAGO3-10 decomposition, which carries the paper, is unaffected.

Label defects. YAGO3-10 negatives are generated, so a negative may be an unrecorded true fact; this inflates the real-name arm and makes our memorisation figure an upper bound with a known direction of error. WN11’s shipped labels include 7 of 10,542 negatives that occur in training, 2,220 duplicate training triples and 193 self-loops. The tuned WN11 run had not converged when training stopped, so 0.9265 is a floor and its memorisation share is conservative.

Transductive memorisation is arguably legitimate. If the graph is fixed, memorising is a correct strategy and our objection would be philosophical. The familiarity split converts it into a measurement: the “neither seen” bucket is the case in which memorisation cannot help.

5.4 Conclusion and future work

For evaluation, an accuracy figure on a transductive benchmark is not a sufficient report; we propose (accuracy, gap) as the minimum unit. For method design, the productive response is not a larger model but a training signal that makes name lookup insufficient, and condition D is the measurable form of that. For enrichment, our decomposition puts a number on what remains once the name carries no pretrained signal: MRR = 0.1336 against a floor of 0.0900, where the same system reports 0.8169 with names intact. We stop short of transferring that figure to genuinely unseen entities: an anonymised entity still occupies its place in the training graph, whereas an inductive one does not, so 0.1336 is an upper bound on the inductive case rather than an estimate of it. Measuring the gap on an inductive benchmark is the test this motivates, and we have not run it.

We asked what instruction tuning installs when a language model is adapted to knowledge graph completion, and answered it by adding to an early and widely cited pipeline a control it never ran. Replacing entity surface forms with opaque identifiers, while leaving relational structure intact, splits a single accuracy figure into a memorisation term and a residual knowledge term. Evaluated as link prediction on YAGO3-10, 94.0 % of above-chance skill is entity surface form, and a permuted-name control attributes 71.5* points of that to the binding between a name and a node rather than to the readability of the text. A familiarity split places the binding in the pre-trained backbone. Neither exogenous type text nor neighbourhood context recovers what the name supplied.

We do not claim that memorisation is a defect. We claim that it is not what this literature reports itself as doing, that the standard protocol cannot see it, and that reporting an accuracy without a gap is no longer sufficient. The instrument requires one additional training run, reports a ratio rather than a score, and needs no change to the method under test beyond substituting the entity surface forms in its inputs.

Three extensions follow. A scale sweep decides whether this is a statement about small models or about the

paradigm. A second permuted-name arm on a second graph would establish that the **71.5/22.5/6.0*** split is not a property of YAGO3-10. Applying the diagnostic to inductive systems is the sharpest available test: there the gap should be small, and a large one would be a considerably stronger result than the one reported here.

Acknowledgment

[Supervisor and institution. Funding, if any, belongs in an unnumbered footnote on the first page.]

References

- [1] L. Yao, J. Peng, C. Mao, and Y. Luo, “Exploring large language models for knowledge graph completion,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2025.
- [2] L. Yao, J. Peng, C. Mao, and Y. Luo, “Exploring large language models for knowledge graph completion,” arXiv:2308.13916, 2023.
- [3] L. Yao, C. Mao, and Y. Luo, “KG-BERT: BERT for knowledge graph completion,” arXiv:1909.03193, 2019.
- [4] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, “Translating embeddings for modeling multi-relational data,” in *Proc. Advances in Neural Information Processing Systems*, 2013, pp. 2787–2795.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [6] B. Shi and T. Wenginger, “Open-world knowledge graph completion,” in *Proc. AAAI Conf. Artificial Intelligence*, 2018, pp. 1957–1964.
- [7] D. Riva, A. Ferrara, and S. Montanelli, “Open-world knowledge graph completion: A survey,” *ACM Comput. Surv.*, 2026.
- [8] M. Li, C. Yang, C. Xu, Z. Song, X. Jiang, J. Guo, H.-f. Leung, and I. King, “Latent type constraints for inductive knowledge graph completion,” 2025.
- [9] Y. Zhang, J. Hu, C. Xiong, Z. Qin, H. Chen, and Z. Li, “Relation-constrained inductive knowledge graph completion with large language models,” 2026.
- [10] J. Gong, X. Fang, W. Zhu, J. Peng, L. Wang, and M. Chen, “Knit: Toward alleviating large language model fact knowledge hallucinations in knowledge graph completion,” *Big Data Mining and Analytics*, vol. 9, no. 2, pp. 611–631, 2026.
- [11] P. Zhang, X. Xu, J. Wu, X. Lu, J. Shi, X. Zhang, D. Cui, Z. Ou, M. Song, and Y. Zhu, “Mitigating hallucinations in knowledge graph completion via embedding-guided instruction tuning,” *Information*, vol. 17, no. 2, art. 207, 2026.
- [12] Y. Wei, Q. Huang, Y. Zhang, and J. T. Kwok, “KICGPTv2: Large language model with knowledge in context for knowledge graph completion,” *IEEE Trans. Knowledge and Data Engineering*, vol. 38, no. 7, 2026.
- [13] X. Song, Y. Wang, B. Zhou, H. Wang, Y. Huang, and L. Gao, “Incorporating structural knowledge into language models for open knowledge graph completion,” *World Wide Web*, vol. 28, art. 8, 2025.
- [14] Z. Wang et al., “Thinking on context: Inductive relation prediction guided by large language models,” *IEEE Trans. Knowledge and Data Engineering*, vol. 37, no. 10, 2025.
- [15] R. Yang, J. Zhu, J. Man, H. Liu, L. Fang, and Y. Zhou, “GS-KGC: A generative subgraph-based framework for knowledge graph completion with large language models,” *Information Fusion*, 2025.
- [16] J. Zhu and P. De Meo, “Exploring large language models for knowledge graph completion with auto-prompting,” in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2025.
- [17] Z. Zheng, Y. Dong, S. Wang, H. Liu, Q. Wang, and J. Li, “KG-CF: Knowledge graph completion with context filtering under the guidance of large language models,” in *Proc. IEEE Int. Conf. Big Data*, 2024.
- [18] L. Guo, Z. Chen, J. Chen, Y. Fang, W. Zhang, and H. Chen, “MKGL: Mastery of a three-word language,” in *Proc. Advances in Neural Information Processing Systems*, 2024.
- [19] B. Xue, Y. Xu, B. Ma, Y. Song, J. Ding, L. Fu, and X. Wang, “FLAME: Empowering frozen LLMs for knowledge graph completion,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 20366–20370.
- [20] Y. Yuan, Y. Xu, and W. Zhang, “Is large language model good at triple set prediction? An empirical study,” 2024.
- [21] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020.
- [22] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [23] F. M. Suchanek, G. Kasneci, and G. Weikum, “YAGO: A core of semantic knowledge,” in *Proc. Int. Conf. World Wide Web (WWW)*, 2007, pp. 697–706.
- [24] A. Yang et al., “Qwen2.5 technical report,” arXiv:2412.15115, 2024.